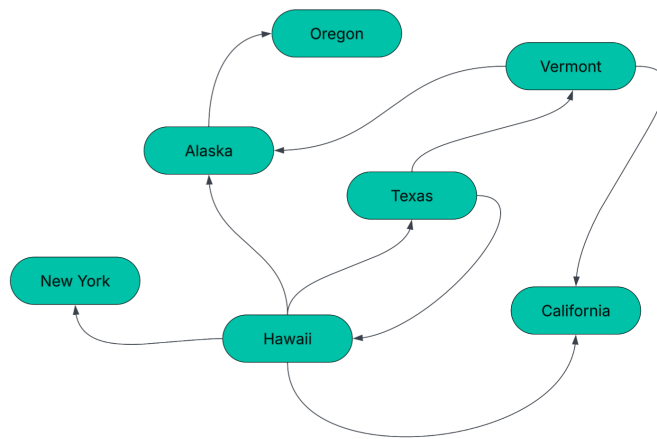


CMSC204
Kartchner

$V(\text{StateGraph}) = \{\text{Oregon, Alaska, Texas, Hawaii, Vermont, New York, California}\}$

$E(\text{StateGraph}) = \{(\text{Alaska, Oregon}), (\text{Hawaii, Alaska}), (\text{Hawaii, Texas}), (\text{Texas, Hawaii}), (\text{Hawaii, California}), (\text{Hawaii, New York}), (\text{Texas, Vermont}), (\text{Vermont, California}), (\text{Vermont, Alaska})\}$

1. Draw the StateGraph



1. Describe the graph pictured above, using the formal graph notation.

$G = (V, E)$

$V(\text{StateGraph}) = \{\text{Oregon, Alaska, Texas, Hawaii, Vermont, New York, California}\}$

$E(\text{StateGraph}) = \{(\text{Alaska, Oregon}), (\text{Hawaii, Alaska}), (\text{Hawaii, Texas}), (\text{Texas, Hawaii}), (\text{Hawaii, California}), (\text{Hawaii, New York}), (\text{Texas, Vermont}), (\text{Vermont, California}), (\text{Vermont, Alaska})\}$

2. a. Is there a path from Oregon to any other state in the graph?

No

b. Is there a path from Hawaii to every other state in the graph?

Yes

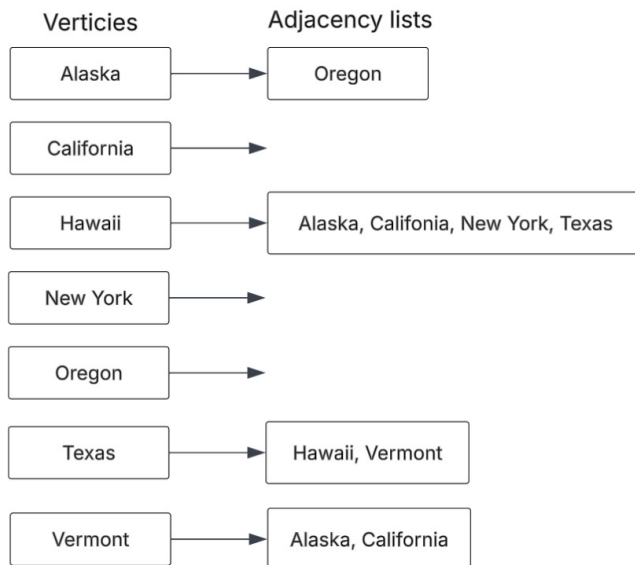
c. From which state(s) in the graph is there a path to Hawaii?

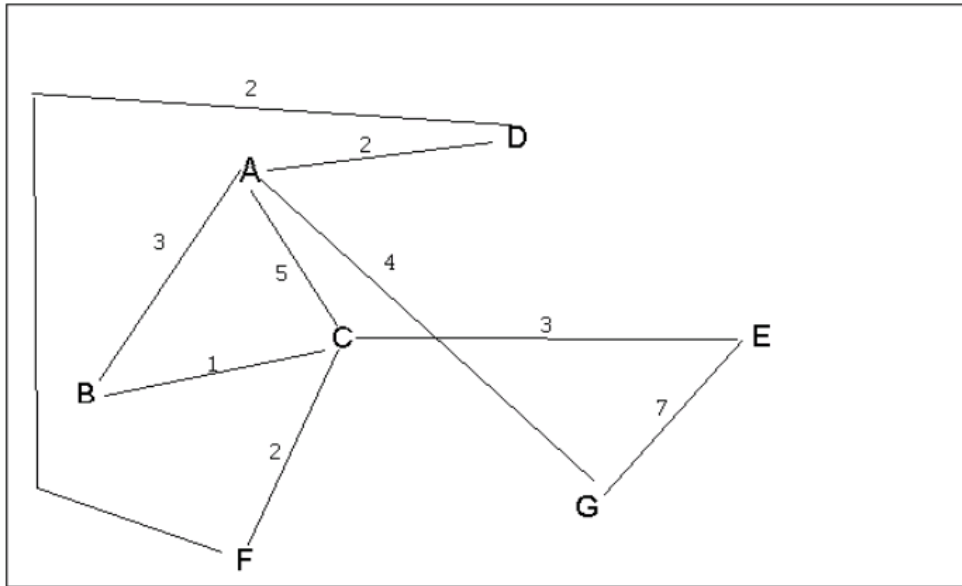
Texas and Hawaii

3. a. Show the adjacency matrix that would describe the edges in the graph.
Store the vertices in alphabetical order

	Alaska	California	Hawaii	New York	Oregon	Texas	Vermont
Alaska	0	0	0	0	1	0	0
California	0	0	0	0	0	0	0
Hawaii	1	1	0	1	0	1	0
New York	0	0	0	0	0	0	0
Oregon	0	0	0	0	0	0	0
Texas	0	0	1	0	0	0	1
Vermont	1	1	0	0	0	0	0

3. b. Show the adjacency lists
that would describe the edges in the graph



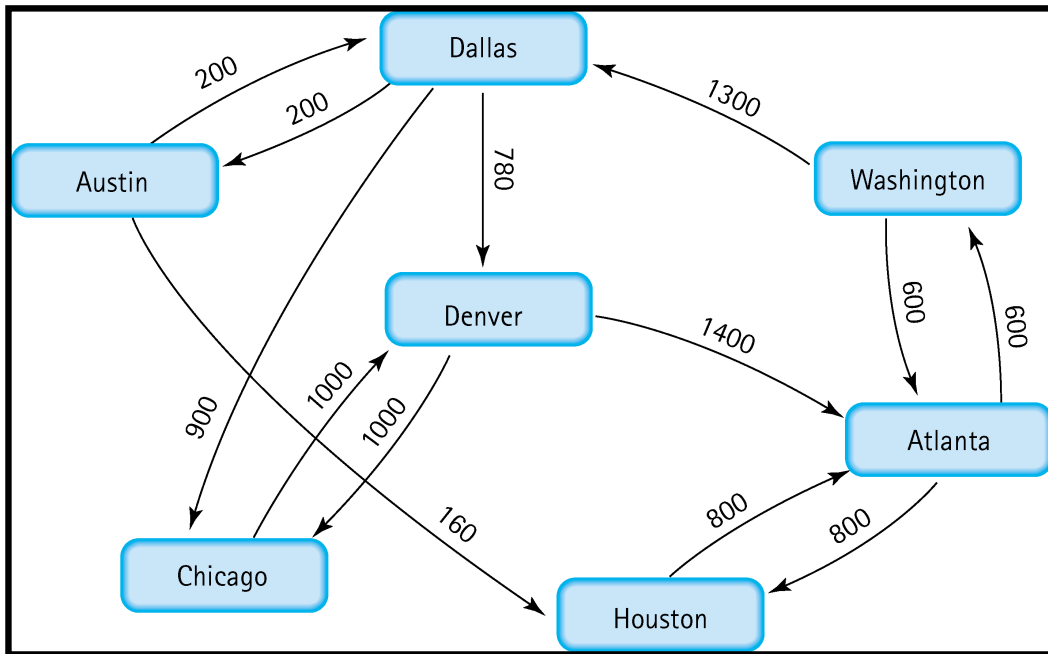


4 a. Which of the following lists the graph nodes in depth first order beginning with E?

- A) E, G, F, C, D, B, A
- B) G, A, E, C, B, F, D
- C) E, G, A, D, F, C, B**
- D) E, C, F, B, A, D, G

4 b. Which of the following lists the graph nodes in breadth first order beginning at F?

- A) F, C, D, A, B, E, G**
- B) F, D, C, A, B, C, G
- C) F, C, D, B, G, A, E
- D) a, b, and c are all breadth first traversals



5. Find the shortest distance from Atlanta to every other city

Atlanta: 0

Washington: 600

Houston: 800

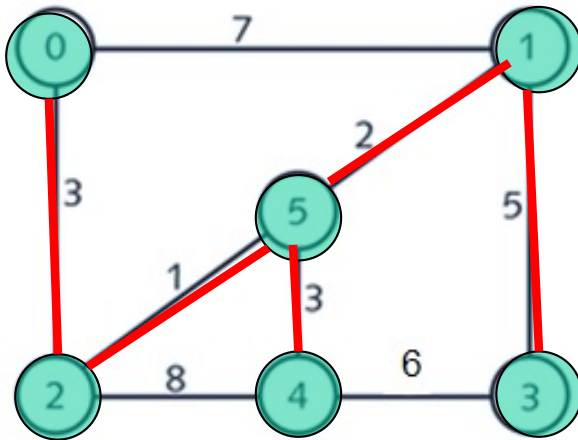
Dallas: 1900 (Atlanta -> Washington -> Dallas)

Austin: 2100 (Atlanta -> Washington -> Dallas -> Austin)

Denver: 2680 (Atlanta -> Washington -> Dallas -> Denver)

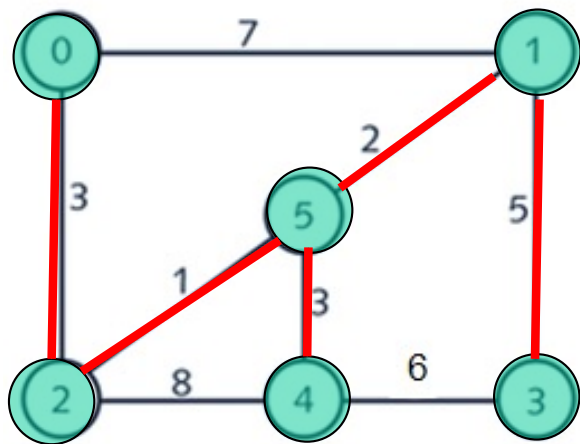
Chicago: 2800 (Atlanta -> Washington -> Dallas -> Chicago)

6. Find the minimal spanning tree using Prim's algorithm. Use 0 as the source vertex . Show the steps.



Step	Choose edge	Weight	Vertices included
Start	-		{0}
1	0 - 2	3	{0,2}
2	2 - 5	1	{0,2,5}
3	5 - 1	2	{0,2,5,1}
4	5 - 4	3	{0,2,5,1,4}
5	1 - 3	5	{0,2,5,1,4,3}

7. Find the minimal spanning tree using Kruskal’s algorithm. Show the weights in order and the steps.



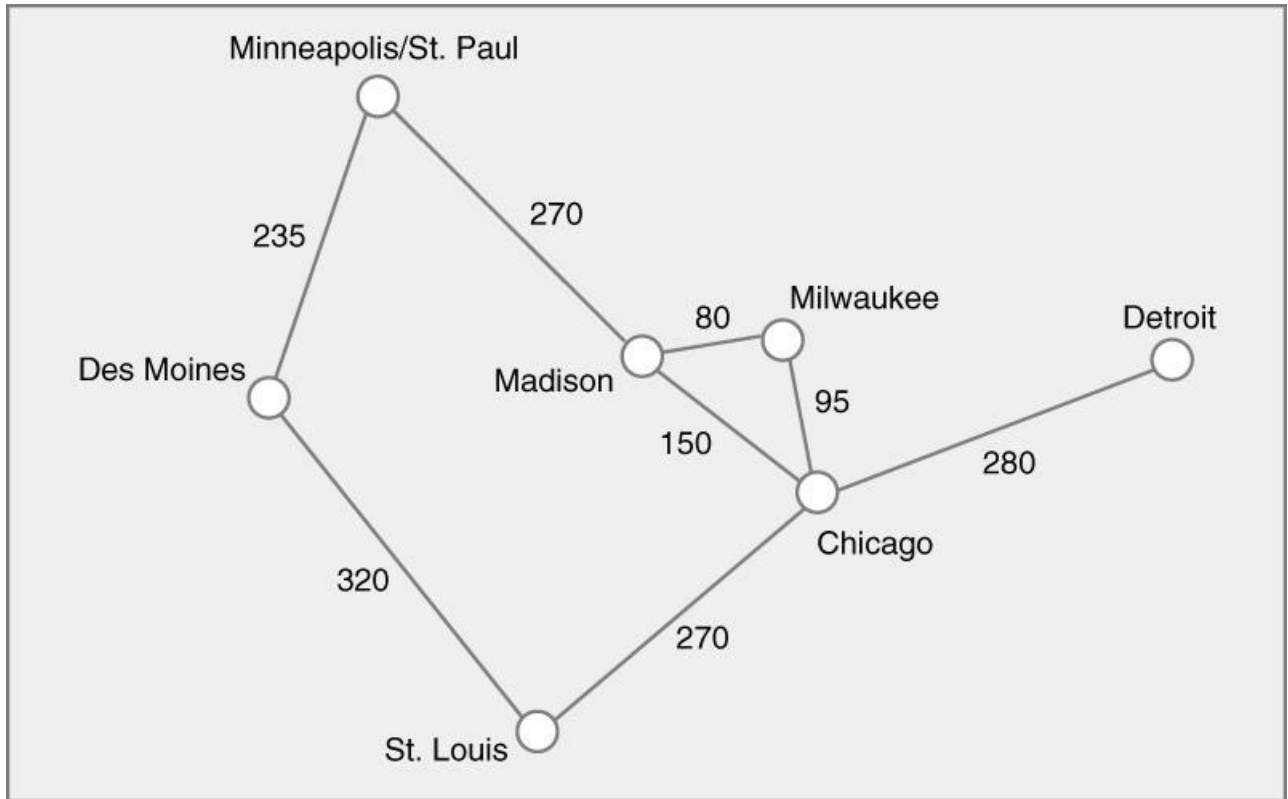
List all edges by weight from smallest to largest

Order	Edge	Weight
1	2 - 5	1
2	1 - 5	2
3	0 - 2	3
4	4 - 5	3
5	1 - 3	5
6	3 - 4	6
7	0 - 1	7
8	2 - 4	8

Choose edges in order, skipping any edge that makes a cycle

Step	Edge	Use?	Reason
1	2 - 5	yes	smallest
2	1 - 5	yes	no cycle
3	0 - 2	yes	no cycle
4	4 - 5	yes	no cycle
5	1 - 3	yes	no cycle
All 6 vertices are connected, so stop			

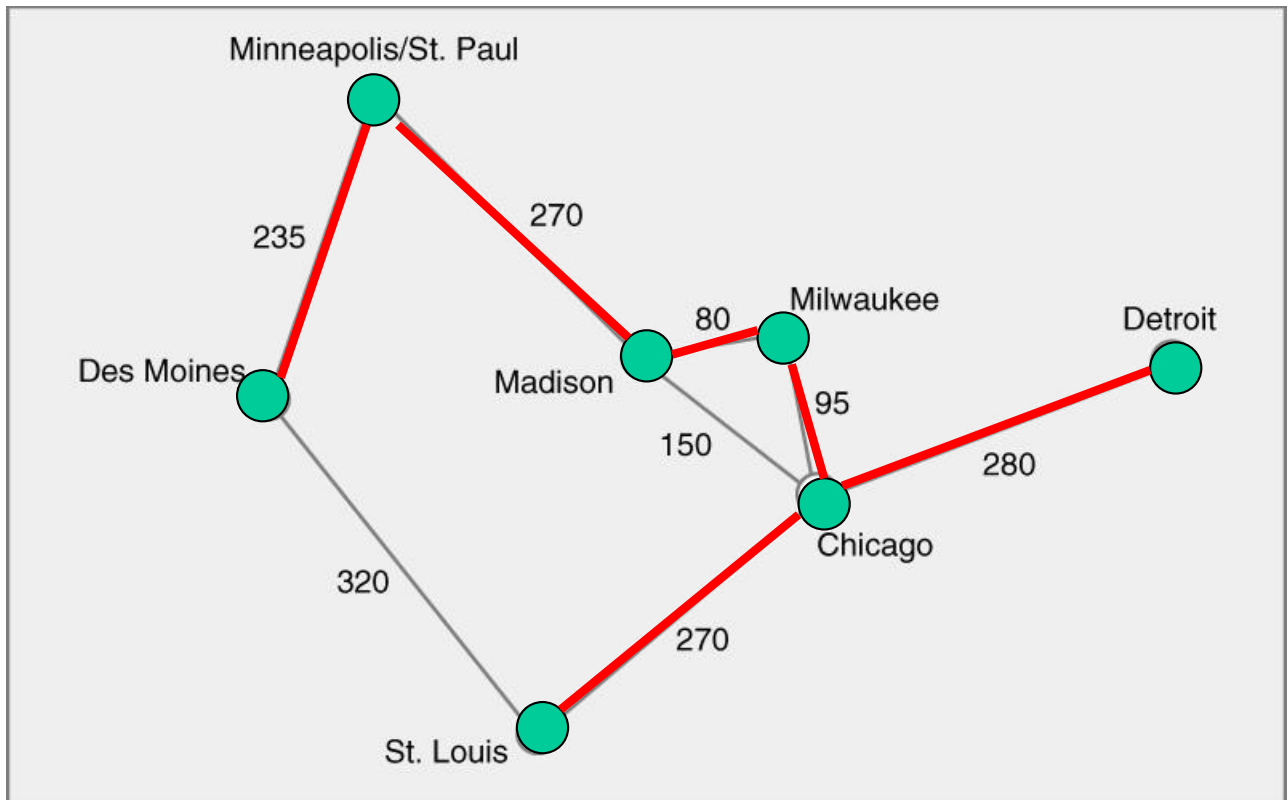
8. Find the minimal spanning tree using the algorithm you prefer. Use Minneapolis/St. Paul as the source vertex



Kruskal's algorithm

List all edges by weight from smallest to largest

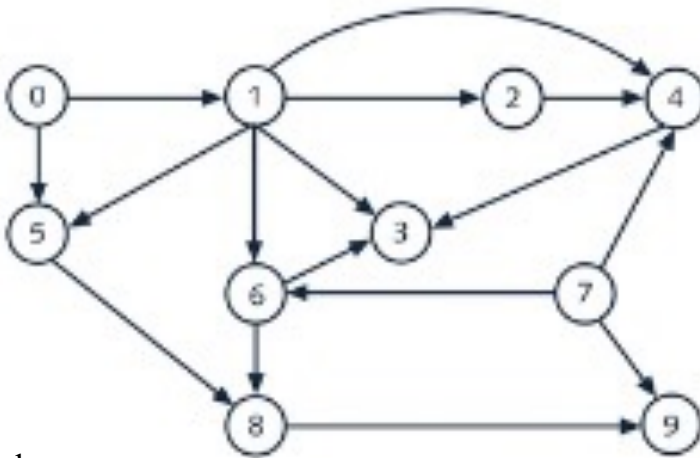
Order	Edge	Weight
1	Madison - Milwaukee	80
2	Chicago - Milwaukee	95
3	Chicago - Madison	150
4	Des Moines – Minneapolis/St. Paul	235
5	Chicago – St. Louis	270
6	Madison - Minneapolis/St. Paul	270
7	Chicago - Detroit	280
8	Des Moines – St. Louis	320



Choose edges in order, skipping any edge that makes a cycle

step	Edge	Use?	Reason
1	Madison - Milwaukee	yes	smallest
2	Chicago - Milwaukee	yes	no cycle
3	Chicago - Madison	no	cycle
4	Des Moines – Minneapolis/St. Paul	yes	no cycle
5	Chicago – St. Louis	yes	no cycle
6	Madison - Minneapolis/St. Paul	yes	no cycle
7	Chicago - Detroit	280	no cycle
8	Des Moines – St. Louis	no	Done

9. List the nodes of the graph in a breadth first topological ordering. Show the steps using arrays predCount, topologicalOrder and a queue



Initial:

0	1	2	3	4	5	6	7	8	9	
0	1	1	3	3	2	2	0	2	2	predCount

Queue = [0, 7]

Step 1: Remove 0

Add 0 to order.

0 points to 1 and 5, so subtract 1 from their predCount

1: 1 -> 0 add to queue

5: 2 -> 1

Queue = [7, 1]

Order = 0

0	1	2	3	4	5	6	7	8	9	
0	0	1	3	3	1	2	0	2	2	predCount

Step 2: Remove 7

Add 7 to order.

7 points to 4, 6 and 9, so subtract 1 from their predCount

4: 3 -> 2

6: 2 -> 1

9: 2 -> 1

Queue = [1]

Order = 0, 7

0	1	2	3	4	5	6	7	8	9	
0	0	1	3	2	1	1	0	2	1	predCount

Step 3: Remove 1

Add 1 to order.

1 points to 2, 3, 5, and 6, so subtract 1 from their predCount

2: 1 -> 0 add 2

3: 3 -> 2

4: 2 -> 1

5: 1 -> 0 add 5

6: 1 -> 0 add 6

Queue = [2, 5, 6]

Order = 0, 7, 1

0	1	2	3	4	5	6	7	8	9	
0	0	0	2	1	0	0	0	2	1	predCount

Step 4: Remove 2

Add 2 to order.

2 points to 4 so subtract 1 from their predCount

4: 1 -> 0 add 4

Queue = [5, 6, 4]

Order = 0, 7, 1, 2

0	1	2	3	4	5	6	7	8	9	
0	0	0	2	0	0	0	0	2	1	predCount

Step 5: Remove 5

Add 5 to order.

5 points to 8, so subtract 1 from their predCount

8: 2 -> 1

Queue = [6, 4]

Order = 0, 7, 1, 2, 5

0	1	2	3	4	5	6	7	8	9	
0	0	0	2	0	0	0	0	1	1	predCount

Step 6: Remove 6

Add 6 to order.

6 points to 3 and 8, so subtract 1 from their predCount

3: 2 -> 1

8: 1 -> 0 add 8

Queue = [4, 8]

Order = 0, 7, 1, 2, 5, 6

0	1	2	3	4	5	6	7	8	9	
0	0	0	1	0	0	0	0	0	1	predCount

Step 7: Remove 4

Add 4 to order.

4 points to 3, so subtract 1 from their predCount

3: 1 -> 0 add 3

Queue = [8, 3]

Order = 0, 7, 1, 2, 5, 6, 4

0	1	2	3	4	5	6	7	8	9	
0	0	0	0	0	0	0	0	0	1	predCount

Step 8: Remove 8

Add 8 to order.

8 points to 9, so subtract 1 from their predCount

9: 1 -> 0 add 9

Queue = [3, 9]

Order = 0, 7, 1, 2, 5, 6, 4, 8

0	1	2	3	4	5	6	7	8	9	
0	0	0	0	0	0	0	0	0	0	predCount

Step 9: Remove 3

Add 3 to order.

3 does not points to any new node

Queue = [9]

Order = 0, 7, 1, 2, 5, 6, 4, 8, 3

0	1	2	3	4	5	6	7	8	9	
0	0	0	0	0	0	0	0	0	0	predCount

Step 10: Remove 9

Add 9 to order.

9 does not points to any new node

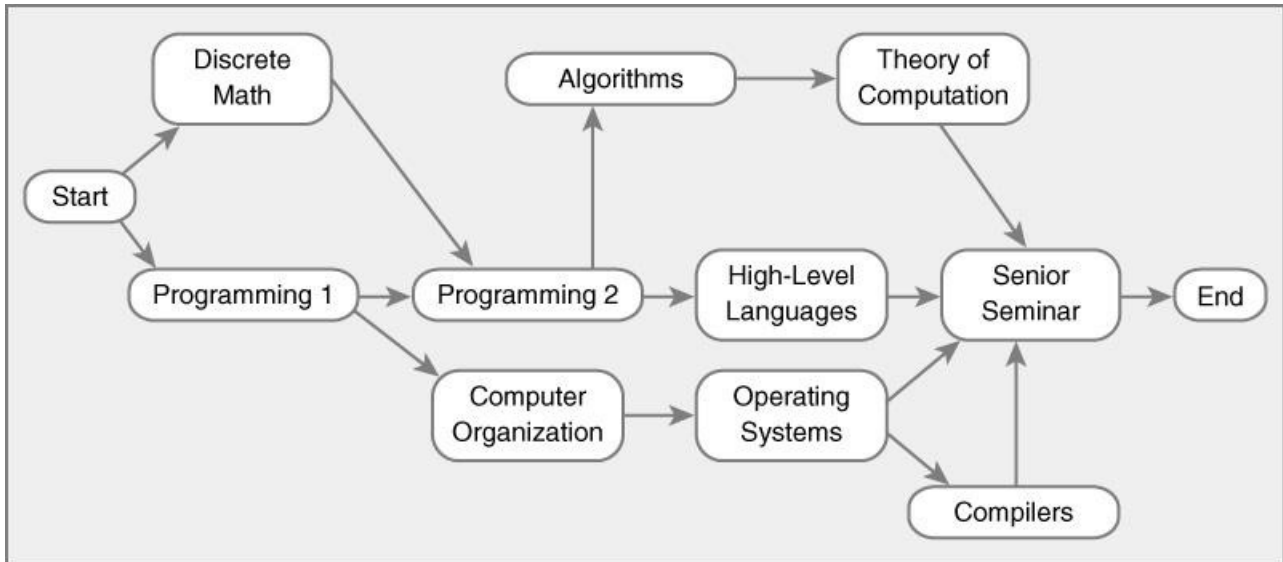
Queue = []

Order = 0, 7, 1, 2, 5, 6, 4, 8, 3, 9

0	1	2	3	4	5	6	7	8	9	
0	0	0	0	0	0	0	0	0	0	predCount

Final topological order: 0, 7, 1, 2, 5, 6, 4, 8, 3, 9

10. List the nodes of the graph in a breadth first topological ordering.



Topological order:

Start,

Discrete Math,

Programming 1,

Programming 2,

Computer Organization,

Algorithms,

High-Level Languages,

Operating Systems,

Theory of Computation,

Compilers,

Senior Seminar,

End